Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Ağ Programlamaya Giriş

Bölüm 12 – Hafta 12

Yazılım Geliştirme II (BILB2000)

Bahar Dönemi – 12. Hafta

Bu Haftanın Planı

1. Ünite 1: Ağ Temelleri
2. Ünite 2: TCP Soket Programlama
3. Ünite 3: Qt Network Modülü
4. Ünite 4: Uygulama Protokolü Tasarımı

Hafta 12'nin Büyük Resmi

OBYS'yi ağa açıyoruz

Bölüm 1-11'de tek makinede çalışan, çok kanallı, responsive bir OBYS kurduk. Bu hafta OBYS'yi **ağa açmaya** hazırlanıyoruz: ağ temelleri, POSIX soketler, Qt Network ve uygulama protokolü tasarımı.

Kazanımlar

- OSI & TCP/IP katman modeli
- TCP vs UDP, port, IP, protokol
- POSIX soket API: `socket`, `bind`, `listen`
- Qt: `QTcpSocket`, `QTcpServer`
- Mesaj çerçeveleme stratejileri
- JSON tabanlı uygulama protokolü

OBYS Bağlantısı

- Veriler artık merkezi sunucuda
- Çoklu istemci aynı anda
- Echo sunucu prototipi
- OBYS Protokol v1 taslağı
- 13. hafta: gerçek OBYS sunucusu
- Hâlâ thread-safe Depo

Ünite 1: Ağ Temelleri

Ünite 1 – Öğrenme Hedefleri

Bu ünitede öğreneceğin

- Ağ neden katmanlı tasarlanır? Kapsülleme nedir?
- OSI 7 katman ve TCP/IP 4 katman karşılaştırması
- IP, port, protokol kavramları
- TCP ve UDP farkları, hangisi ne zaman?
- Üç-yollu el sıkışma (three-way handshake)
- İstemci-sunucu mimarisi
- Bayt sırası (endianness) ve ağ sırası

Ağ Niye Katmanlı?

Karmaşıklığı bölmek

Ağ üzerinden veri gönderirken pek çok sorun çözülmeli: fiziksel sinyalleşme, hata düzeltme, yönlendirme, akış kontrolü, uygulama anlamı. Hepsini tek katmanda çözmek imkânsız.

Kapsülleme (encapsulation)

Her katman bir altındakini kullanır, kendi başlığını ekler ve bir üstüne hizmet sunar. Üstteki katman alttakinin nasıl çalıştığını bilmek zorunda değil.

Analoji: Posta sistemi

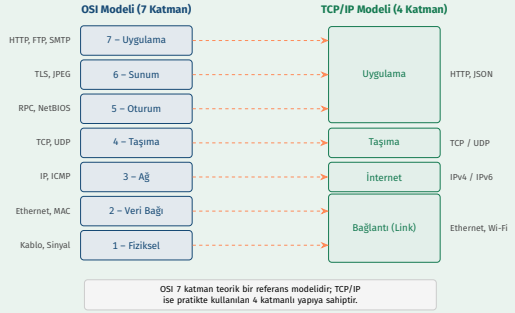
Mektup (uygulama) zarfa girer (taşıma), zarf çantaya (ağ), çanta kamyona (fiziksel). Her aşamada üstüne yeni etiket eklenir.

OSI 7 Katman Modeli

Katmanlar (yukarıdan aşağı)

#	Katman	Örnek
7	Uygulama	HTTP, FTP, SMTP
6	Sunum	TLS, JPEG, MIME
5	Oturum	NetBIOS, RPC
4	Taşıma	TCP, UDP
3	Ağ	IP, ICMP
2	Veri Bağı	Ethernet, Wi-Fi
1	Fiziksel	Kablo, RF

Görsel



Pratikte

OSI bir referans modelidir. Gerçek dünyada TCP/IP daha yaygın.

TCP/IP 4 Katman Modeli

TCP/IP karşılığı

#	TCP/IP	OSI Eşi	Protokol
4	Uygulama	5-6-7	HTTP, DNS, SMTP, FTP
3	Taşıma	4	TCP, UDP
2	İnternet	3	IP, ICMP, ARP
1	Ağ Erişimi	1-2	Ethernet, Wi-Fi, PPP

Bizim ilgi alanımız

Bu hafta **taşıma katmanı (TCP)** ve **uygulama katmanı** (kendi protokolümüz). Alt katmanlar işletim sistemi tarafından bizim için halledilir.

IP, Port, Protokol Üçlüsü

IP Adresi

- Bir makineyi tanımlar
- IPv4: `192.168.1.42`
- IPv6: `::1`
- `127.0.0.1` = localhost

Port

- Makinedeki uygulamayı seçer
- 0-65535 arası
- 0-1023 ayrıcalıklı
- OBYS örnek: 5500

Protokol

- İletişim kuralları
- TCP, UDP
- Veri formatı
- Komut/cevap düzeni

Üçü birlikte

Bir uç noktayı tam tanımlamak için üçü gerekir: `tcp://192.168.1.42:5500` (protokol + IP + port)

TCP vs UDP

Özellik	TCP	UDP
Bağlantı	Bağlantılı	Bağlantısız
Güvenilirlik	Garantili teslim	Best-effort
Sıra	Sıralı paketler	Sırasız olabilir
Akış kontrolü	Var	Yok
Tıkanıklık kontrolü	Var	Yok
Üst yük	Yüksek (başlık 20+ B)	Düşük (başlık 8 B)
Veri modeli	Akış (stream)	Mesaj (datagram)
Tipik kullanım	Web, e-posta, dosya	Video, oyun, DNS

OBYS için seçim

Veri kaybı kabul edilemez (öğrenci kaydı kaybolmamalı), sıra önemli (komutlar peş peşe), bağlantı durumu önemli. **TCP** doğru tercih.

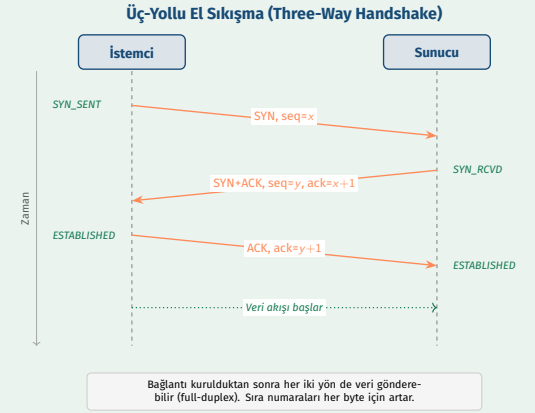
Üç-Yollu El Sıkışma (Three-Way Handshake)

TCP bağlantı kurulumu

1. İstemci → Sunucu: `SYN` (seq=x)
2. Sunucu → İstemci: `SYN+ACK` (seq=y, ack=x+1)
3. İstemci → Sunucu: `ACK` (ack=y+1)

Bu üç paket sonunda iki taraf da seq numaralarında anlaşmış olur.

Görsel



Neden 3 paket?

İstemci-Sunucu Mimarisi

Sunucu (passive)

- Bilinen bir port'ta bekler
- Bağlantı isteklerini kabul eder
- Birden çok istemci ile aynı anda
- Genelde uzun ömürlü

İstemci (active)

- Sunucuya bağlantı başlatır
- Adres + port bilmesi gerek
- Çoğunlukla kısa ömürlü
- Bir veya birkaç sunucu ile

Asimetrik roller, simetrik veri

Bağlantı kurulduktan sonra her iki taraf da **aynı anda** veri gönderip alabilir. TCP bağlantısı çift yönlüdür (full duplex).

Bayt Sırası: htons / htonl Tuzağı

Endianness sorunu

Bilgisayarlar çok baytlı sayıları farklı sıralayabilir: **little-endian** (x86, ARM) düşük baytı önce; **big-endian** ise yüksek baytı önce. Ağda hep big-endian (network byte order) kullanılır.

```
#include <arpa/inet.h>

uint16_t port = 5500;
uint16_t ag_port = htons(port);   // host -> network short

uint32_t ip = INADDR_ANY;
uint32_t ag_ip = htonl(ip);       // host -> network long

// Geri donus:
uint16_t yerel = ntohs(ag_port);  // network -> host short
```

Unutursan

Port 5500 yerine 35861 dinlersin; istemci hiç bağlanamaz, sebebi gözükmez. **Her zaman** `htons/htonl` kullan.

Ünite 2: TCP Soket Programlama

Ünite 2 – Öğrenme Hedefleri

Bu ünitede öğreneceğin

- POSIX soket API'sinin genel görünümü
- `socket`, `bind`, `listen`, `accept` çağrıları
- `SO_REUSEADDR` neden zorunlu?
- Bloklayıcı I/O ve `recv` döngüsü
- Echo sunucu ve echo istemci yazımı
- `connect` hata kodları (ECONNREFUSED, ETIMEDOUT)
- TCP durum diyagramı

POSIX Soket API: Genel Görünüm

Soket nedir?

Socket, ağ uç noktasını temsil eden bir dosya tanıtıcısıdır (file descriptor). UNIX felsefesi: **her şey dosyadır**, ağ bağlantısı dahil.

Çağrı	İşlev
`socket()`	Yeni soket oluştur
`bind()`	Soketi yerel adres/port'a bağla
`listen()`	Bağlantıları kabul edilebilir hale getir
`accept()`	Gelen bağlantıyı kabul et
`connect()`	Uzak adrese bağlan
`send()` / `recv()`	Veri gönder / al
`close()`	Soketi kapat

socket() Çağrısı

```c
#include <sys/socket.h>
#include <netinet/in.h>

int sd = socket(AF_INET,          // Adres ailesi: IPv4
                SOCK_STREAM,      // Soket tipi: TCP (stream)
                0);               // Protokol: 0 = otomatik

if (sd < 0) {
    perror("socket");
    return 1;
}
```

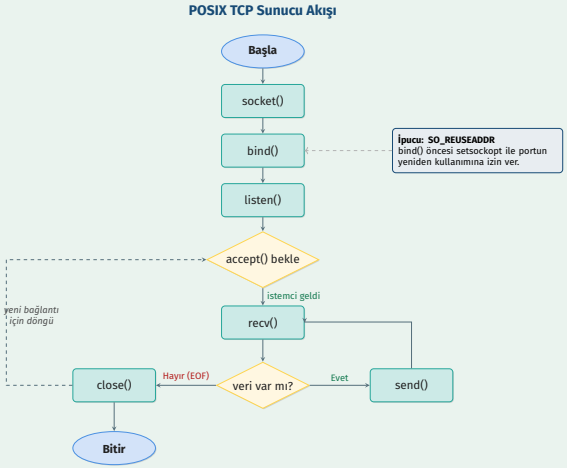
- AF_INET = IPv4, AF_INET6 = IPv6
- SOCK_STREAM = TCP, SOCK_DGRAM = UDP
- Dönüş değeri: dosya tanıtıcısı veya -1 (hata)
- errno hata sebebini taşır

Sunucu Yaşam Döngüsü

Adımlar

1. `socket()`
2. `setsockopt(SO_REUSEADDR)`
3. `bind()`
4. `listen()`
5. Döngü: `accept()`
6. Her bağlantı için: `recv/send`
7. `close()`

Görsel



bind() ve SO_REUSEADDR Zorunluluğu

```
int evet = 1;
setsockopt(sd, SOL_SOCKET, SO_REUSEADDR, &evet, sizeof(evet));

sockaddr_in adres{};
adres.sin_family      = AF_INET;
adres.sin_addr.s_addr = htonl(INADDR_ANY);   // Tum arayuzlerde
adres.sin_port        = htons(5500);

if (bind(sd, (sockaddr*)&adres, sizeof(adres)) < 0) {
    perror("bind"); return 1;
}
```

SO_REUSEADDR olmazsa

Sunucu çöktüğünde port `TIME_WAIT` durumunda kalır. Yeniden başlatınca **Address already in use** hatası alırsın ve yaklaşık 60 saniye beklemen gerekir. Geliştirme akışını öldürür.

listen() ve accept()

```
listen(sd, 16);   // 16 = bekleme kuyrugu uzunlugu (backlog)

while (true) {
    sockaddr_in istemci{};
    socklen_t boy = sizeof(istemci);

    int istemci_sd = accept(sd,
                            (sockaddr*)&istemci,
                            &boy);
    // istemci_sd: yeni dosya tanitici, sadece bu istemci icin
    if (istemci_sd < 0) { perror("accept"); continue; }

    isle(istemci_sd);    // recv/send dongusu
    close(istemci_sd);
}
```

Bloklayıcı davranış

accept bağlantı gelene kadar **bloklar**. Tek-thread'li sunucu o sırada başka iş yapamaz. Çözüm: thread, async, select/epoll.

recv() Döngüsü ve EOF

```
char tampon[1024];
while (true) {
    ssize_t n = recv(istemci_sd, tampon, sizeof(tampon), 0);
    if (n == 0) {
        // Karsi taraf baglantiyi kapatti (EOF)
        break;
    }
    if (n < 0) {
        if (errno == EINTR) continue;   // sinyal kesintisi
        perror("recv"); break;
    }
    // n bayt isle
    isle(tampon, n);
}
```

Üç farklı durum

- `n > 0`: Veri var, işle
- `n == 0`: Bağlantı temiz kapandı (EOF)
- `n < 0`: Hata (`errno` kontrol et)

Echo Sunucu (POSIX, basitleştirilmiş)

```
int sunucu = socket(AF_INET, SOCK_STREAM, 0);
int evet = 1;
setsockopt(sunucu, SOL_SOCKET, SO_REUSEADDR, &evet, sizeof(evet));

sockaddr_in adr{AF_INET, htons(5500), {htonl(INADDR_ANY)}};
bind(sunucu, (sockaddr*)&adr, sizeof(adr));
listen(sunucu, 16);

while (true) {
    int istemci = accept(sunucu, nullptr, nullptr);
    char buf[1024];
    ssize_t n;
    while ((n = recv(istemci, buf, sizeof(buf), 0)) > 0) {
        send(istemci, buf, n, 0);   // ECHO: aynisini geri gonder
    }
    close(istemci);
}
```

İstemci Yaşam Döngüsü

Adımlar

1. `socket()` – yeni soket
2. `connect()` – sunucuya bağlan
3. Veri alışverişi: `send/recv`
4. `close()`

İstemcide `bind` gerekmez

Çekirdek (kernel) otomatik olarak boş bir port atar (ephemeral port). İstemci hangi porttan çıktığını çoğu zaman önemsemez.

connect() Hataları

```
sockaddr_in hedef{};
hedef.sin_family = AF_INET;
hedef.sin_port   = htons(5500);
inet_pton(AF_INET, "127.0.0.1", &hedef.sin_addr);

if (connect(sd, (sockaddr*)&hedef, sizeof(hedef)) < 0) {
    perror("connect");
}
```

errno	Anlamı
ECONNREFUSED	Karşı tarafta dinleyen yok
ETIMEDOUT	Yanıt yok, zaman aşımı
EHOSTUNREACH	Yönlendirme yok
ENETUNREACH	Ağ erişilemez

Tipik tuzak

ECONNREFUSED = sunucu çalışmıyor. ETIMEDOUT = ağ problemi veya güvenlik duvarı paketi düşürüyor.

Echo İstemci (POSIX)

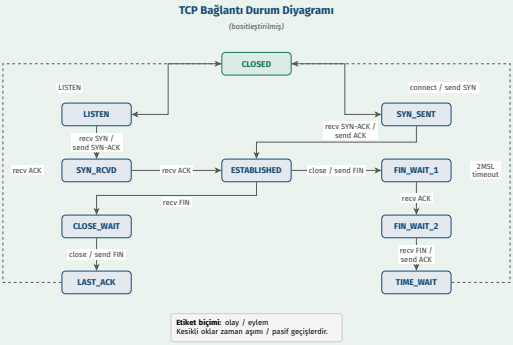
```
int sd = socket(AF_INET, SOCK_STREAM, 0);

sockaddr_in hedef{AF_INET, htons(5500), {}};
inet_pton(AF_INET, "127.0.0.1", &hedef.sin_addr);

if (connect(sd, (sockaddr*)&hedef, sizeof(hedef)) < 0) {
    perror("connect"); return 1;
}

const char* mesaj = "Merhaba sunucu!\n";
send(sd, mesaj, std::strlen(mesaj), 0);

char yanit[256];
ssize_t n = recv(sd, yanit, sizeof(yanit) - 1, 0);
yanit[n] = '\0';
std::cout << "Sunucudan: " << yanit;

close(sd);
```

TCP Bağlantı Durum Diyagramı

Önemli durumlar

- `LISTEN`: Sunucu bekliyor
- `SYN_SENT`: İstemci SYN attı
- `ESTABLISHED`: Bağlantı kuruldu
- `FIN_WAIT_1/2`: Aktif kapatma
- `CLOSE_WAIT`: Pasif kapatma
- `TIME_WAIT`: Kapatan tarafta 2×MSL

Görsel



TIME_WAIT

Kapatan taraf yaklaşık 60 sn bu durumda kalır. Geç gelen paketleri absorbe eder. Bu yüzden `SO_REUSEADDR` hayat kurtarır.

Ünite 3: Qt Network Modülü

Ünite 3 – Öğrenme Hedefleri

Bu ünitede öğreneceğin

- POSIX'ten Qt'ye geçiş motivasyonu
- `QTcpSocket` sinyalleri ve sahiplik
- `connectToHost` ve asenkron bağlantı
- `readyRead` sinyali ve veri okuma
- `QTcpServer` ve `newConnection`
- `nextPendingConnection` kullanımı
- `waitFor*` fonksiyonlarının riskleri
- Çoklu istemci sahiplik modeli

POSIX'ten Qt'ye: Neden Sinyal/Slot?

POSIX yaklaşımı

- Bloklayıcı çağrılar
- Her istemci için thread
- Manuel `select/poll`
- Platform farklılıkları
- GUI ile entegrasyon zor

Qt yaklaşımı

- Asenkron, sinyal tabanlı
- Tek thread, çok istemci
- Otomatik event-loop
- Platform bağımsız
- GUI ile doğal entegrasyon

Felsefe

"`recv` bekleme yapma, **readyRead** sinyaliyle uyandır beni"

QTcpSocket Mirası ve Sinyalleri

Sınıf hiyerarşisi

`QObject` → `QIODevice` → `QAbstractSocket` → `QTcpSocket`

Sinyal	Anlamı
`connected`	Bağlantı kuruldu
`disconnected`	Bağlantı kapandı
`readyRead`	Okunmaya hazır veri var
`bytesWritten(qint64)`	Veri tampondan gönderildi
`errorOccurred(SocketError)`	Hata oluştu
`stateChanged(SocketState)`	Durum değişti

QIODevice avantajı

`QTcpSocket`, dosya gibi davranır: `read/write/readLine` aynı API.

connectToHost ve readyRead Örneği

```
auto* soket = new QTcpSocket(this);

connect(soket, &QTcpSocket::connected, this, [soket] {
    soket->write("Merhaba sunucu\n");
});

connect(soket, &QTcpSocket::readyRead, this, [soket] {
    QByteArray veri = soket->readAll();
    qDebug() << "Sunucudan:" << veri;
});

connect(soket, &QTcpSocket::errorOccurred, this,
    [](QAbstractSocket::SocketError e) {
        qWarning() << "Soket hatasi:" << e;
    });

soket->connectToHost("127.0.0.1", 5500);
```

Akış

`connectToHost` hemen döner. Bağlantı hazır olunca `connected` sinyali gelir. Veri gelince `readyRead`.

Yeni connect Syntax

```
// DOGRU: Yeni syntax (derleme zamani kontrolu)
connect(soket, &QTcpSocket::readyRead,
        this,  &Istemci::veriOku);

// Lambda da olur:
connect(soket, &QTcpSocket::connected, this, [this] {
    durumGosterge_->setText("Bagli");
});

// YANLIS: Eski SIGNAL/SLOT makro syntax'i
connect(soket, SIGNAL(readyRead()), this, SLOT(veriOku()));
```

Yeni syntax avantajları

- Derleme zamanında imza kontrolü
- Lambda desteği
- Otomatik tip dönüşümü
- IDE'de tıklanabilir referans

`waitFor*` Neden Tehlikeli?

waitForReadyRead, waitForConnected, waitForBytesWritten

Bunlar bloklayıcı çağrılardır. Çağrıldığı thread'i durdururlar.

Dikkat! UI thread'inde KULLANMA

- GUI donar
- Event loop işlemez
- Diğer sinyaller kuyruğa düşer
- Kullanıcı pencereyi hareket bile ettiremez
- İptal mekanizması çalışmaz

Sadece şu durumda kullan

Konsol uygulaması veya ayrı bir worker thread içinde, kısa testler veya senkron protokol gereksinimi varsa.

QTcpServer::listen ve newConnection

```
auto* sunucu = new QTcpServer(this);

connect(sunucu, &QTcpServer::newConnection,
        this,   &SunucuPencere::yeniBaglanti);

if (!sunucu->listen(QHostAddress::Any, 5500)) {
    qCritical() << "Listen basarisiz:" << sunucu->errorString();
    return;
}
qDebug() << "Sunucu" << sunucu->serverPort() << "portunda dinliyor";
```

- `QHostAddress::Any` – tüm arayüzler (POSIX `INADDR_ANY`)
- `QHostAddress::LocalHost` – sadece 127.0.0.1
- `listen` otomatik `bind+listen` yapar, SO_REUSEADDR uygular
- Yeni istemci geldiğinde `newConnection` sinyali tetiklenir

nextPendingConnection ve Sahiplik

```
void SunucuPencere::yeniBaglanti() {
    while (sunucu_->hasPendingConnections()) {
        QTcpSocket* istemci = sunucu_->nextPendingConnection();
        // Sahiplik: bizde, ama istemci QObject; parent'i sunucu_
        // disconnected sinyalinde otomatik silinmesi icin:
        connect(istemci, &QTcpSocket::disconnected,
                istemci, &QObject::deleteLater);

        connect(istemci, &QTcpSocket::readyRead, this, [this, istemci] {
            mesajIsle(istemci);
        });
    }
}
```

Sahiplik tuzağı

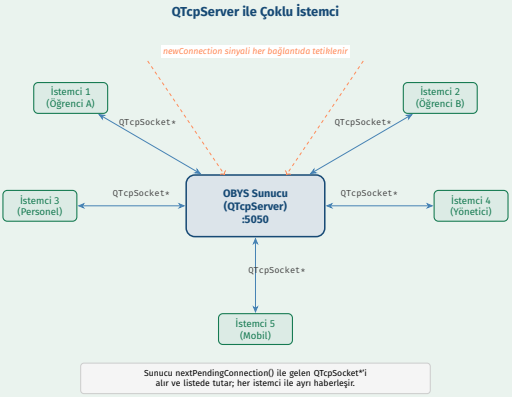
`nextPendingConnection` parent'sız bir QObject döner aslında! Sızıntı olmasın diye `deleteLater` bağlantısı şart.

Çoklu İstemci Modeli

Tek thread, çok soket

- `QTcpServer` her istemci için yeni `QTcpSocket` verir
- Hepsi aynı event loop'ta
- Sinyal/slot ile sıralanır
- Yarış koşulu yok (aynı thread)
- Yüzlerce istemci sorunsuz

Görsel



Ölçek limiti

Echo Sunucu (Qt sürümü)

```
class EchoSunucu : public QObject {
    Q_OBJECT
    QTcpServer* sunucu_;
public:
    EchoSunucu(QObject* p = nullptr) : QObject(p) {
        sunucu_ = new QTcpServer(this);
        connect(sunucu_, &QTcpServer::newConnection,
                this,    &EchoSunucu::yeniBaglanti);
        sunucu_->listen(QHostAddress::Any, 5500);
    }
private slots:
    void yeniBaglanti() {
        QTcpSocket* s = sunucu_->nextPendingConnection();
        connect(s, &QTcpSocket::disconnected, s, &QObject::deleteLater);
        connect(s, &QTcpSocket::readyRead, this, [s] {
            s->write(s->readAll());     // ECHO
        });
    }
};
```

Thread Affinity Uyarısı

QObject thread affinity

Her QObject bir thread'e "aittir" (`thread()` metodu). Sinyal/slot bağlantıları otomatik olarak doğru thread'de çalıştırılır (Qt::AutoConnection).

QTcpSocket farklı thread'de oluşturulamaz

`QTcpSocket` oluşturan thread'in **event loop'u** olmalı. Worker thread'de kullanmak istersen `moveToThread` yap ve o thread'in event loop'unu çalıştır.

Pratik kural

UI thread'de basit istemciler tut. Sunucu yoğunlaşırsa worker thread'de bir QObject'i moveToThread et ve soketleri orada oluştur.

Ünite 4: Uygulama Protokolü Tasarımı

Ünite 4 – Öğrenme Hedefleri

Bu ünitede öğreneceğin

- TCP'nin "stream" doğası: mesaj sınırı yok
- Üç çerçeveleme stratejisi (sabit boy, ayraç, length-prefix)
- Akümülatör tampon (accumulator buffer) deseni
- JSON tabanlı uygulama protokolü
- Komut/cevap şeması ve korelasyon ID
- OBYS Protokol v1 taslağı
- Versiyonlama ve geriye uyumluluk

TCP "Stream": Mesaj Sınırı Yok

Kritik gerçek

TCP **bayt akışı** sunar, mesaj değil. Gönderdiğin `send("AB"); send("CD");` karşı tarafta tek `recv` ile `"ABCD"` olarak gelebilir!

Sonuç

Her uygulama protokolü kendi **çerçevelemesini** (framing) tanımlamak zorunda. Aksi halde mesaj sınırını bilemezsin.

UDP farkı

UDP datagram tabanlıdır; gönderdiğin paket aynı sınırda gelir. Ama UDP güvenilir değil. "Ya yağmur, ya elma."

Üç Çerçeveleme Stratejisi

Strateji	Avantaj	Dezavantaj
Sabit boy (örn. 256 B)	Çok basit, hızlı	Esnek değil, israf
Ayraç (\n, NUL)	İnsan okur, basit	İçerikte ayraç olamaz, escape gerek
Length-prefix (4 B uzunluk + payload)	Esnek, etkin, ikili güvenli	Sabit başlık parse'ı gerekir

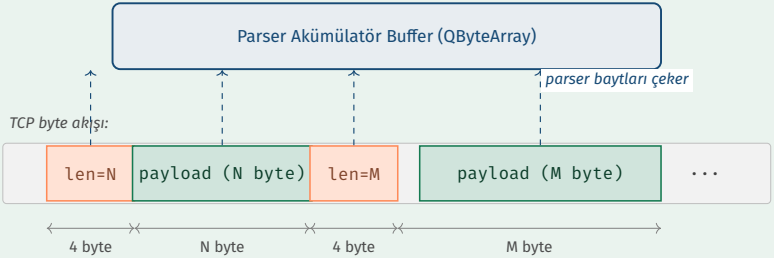
Yaygın seçim

Length-prefix: 4 baytlık big-endian uzunluk, ardından o kadar bayt JSON. HTTP/2, Redis, gRPC hepsi bu aileyi kullanır.

Length-Prefix Çerçeveleme Görseli

Görsel

Length-Prefix Çerçeveleme



Çerçeve biçimi
`[len:4 byte][payload]`
`len = htonl(N)`
N = payload boyutu (byte)

Algoritma: buffer'a gelen byte'ları ekle. Eğer buffer.size() $\geq$ 4 ise len oku; eğer buffer.size() $\geq$ 4+len ise payload'ı ayır ve mesaj olarak işle, buffer'dan kaldır.

Akümülatör Tampon Deseni

```
QByteArray tampon_;   // istemci basina

void readyReadGeldi(QTcpSocket* s) {
    tampon_ += s->readAll();     // her seyi biriktir

    while (true) {
        if (tampon_.size() < 4) return;  // basligi bekle
        quint32 boy = qFromBigEndian<quint32>(
            reinterpret_cast<const uchar*>(tampon_.constData()));
        if (boy > MAX_BOY) { hata("cok buyuk"); s->close(); return; }
        if (tampon_.size() < 4 + (qsizetype)boy) return;  // payload'i bekle

        QByteArray payload = tampon_.mid(4, boy);
        tampon_.remove(0, 4 + boy);
        mesajGeldi(payload);     // tam mesaj
    }
}
```

Mantık

Her `readyRead`'de eklersin, sonra döngüde tam mesaj çıkarılabildiği sürece çıkar. Yarım mesaj kalırsa bekle.

JSON Niye?

JSON avantajları

- İnsan okuyabilir (debug)
- Geniş kütüphane desteği
- Esnek (yeni alan eklenebilir)
- Qt: `QJsonDocument`
- Web ile uyumlu

Dezavantajları

- Metin tabanlı (büyük)
- Parse maliyeti var
- Tip bilgisi sınırlı
- Yorum yok

OBYS için karar

Trafik yoğun değil, debug çok kolay olmalı, esneklik önemli. **JSON over TCP** doğru tercih. (Yüksek hız gerekirse Protobuf/MsgPack.)

JSON Format Örneği

```
// Komut (istemci -> sunucu)
{
  "v": 1,
  "id": "req-42",
  "komut": "ogrenci_ekle",
  "veri": {
    "no": 22001,
    "ad": "Ayse Yilmaz",
    "bolum": "Bilgisayar"
  }
}

// Cevap (sunucu -> istemci)
{
  "v": 1,
  "id": "req-42",
  "durum": "ok",
  "veri": { "ogrenci_id": 5821 }
}
```

Komut/Cevap Şeması

Zorunlu alanlar

Alan	Yön	Anlamı
`v`	Her ikisi	Protokol versiyonu
`id`	Her ikisi	Korelasyon kimliği
`komut`	İstek	İşlem adı
`veri`	Her ikisi	Yük (payload)
`durum`	Cevap	"ok" / "hata"
`hata_kodu`	Cevap (hata)	Sayısal hata kodu
`mesaj`	Cevap (hata)	İnsan okur açıklama

Tutarlı şema

Her komut/cevap aynı kabuk yapısını kullanır. Yeni komut eklerken protokolün geri kalanı değişmez.

Korelasyon ID (Request ID)

Niye gerekli?

Asenkron istemci aynı anda 5 farklı komut gönderebilir. Cevaplar hangi komuta ait? `id` alanı bu eşleşmeyi sağlar.

Pattern: bekleyen istekler haritası

- İstemci: `QHash<QString, Callback> bekleyen_;`
- Komut gönderirken `id` üret, callback kaydet
- Cevap gelince `bekleyen_[id]` ile callback'i bul ve sil

Zaman aşımı koy

Cevap gelmeyen istekleri silmek için zaman aşımı timer'ı şart. Aksi halde harita sızıntı yapar.

OBYS Protokol v1 – Komut Listesi

Komut	Açıklama	Veri Alanları
`ping`	Bağlantı testi	(yok)
`ogrenci_ekle`	Yeni öğrenci kaydı	no, ad, bolum
`ogrenci_bul`	Numaraya göre öğrenci	no
`ogrenci_listele`	Tüm öğrenciler	sayfa, boyut
`ogrenci_sil`	Kayıt sil	no
`ogrenci_guncelle`	Alanları güncelle	no, alan, deger
`ders_ata`	Derse kayıt	ogr_no, ders_kodu
`not_gir`	Sınav notu	ogr_no, ders_kodu, not

Genişleme yolu

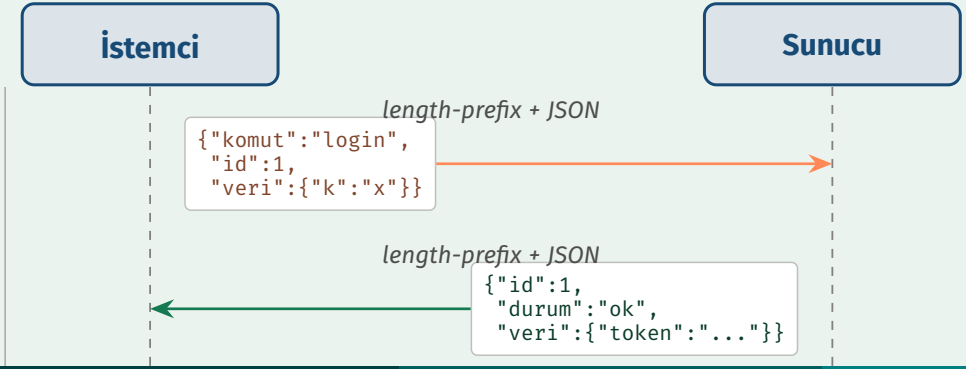
Yeni komut eklemek = JSON'da yeni `komut` değeri. Eski istemciler yeni komutu bilmese de eski komutlar çalışmaya devam eder.

OBYS Protokol Akışı (Görsel)

Görsel

OBYS Komut/Cevap Protokolü

(length-prefix + JSON üzerinden)



Versiyonlama Önemi

`v` alanı boşuna değil

Protokol günün birinde değişecek. **Plan yapmazsan** çakılırsın.

Stratejiler

- **Geriye uyumlu eklemeler**: Yeni alan ekle, eski alanları silme
- **Yumuşak göç**: `v=1` ve `v=2`'yi bir süre paralel destekle
- **Reddetme**: Sunucu desteklemediği versiyonu reddetsin
- **Negotiation**: İlk pakette desteklenen versiyon listesi

Pratik tavsiye

Bilinmeyen alanları görmezden gel (Postel ilkesi: "göndermede sıkı, almada hoşgörülü"). Böylece eski sunucu yeni istemciyle çalışır.

Bu Hafta Neyi Başardık?

Ağ temelleri

- OSI / TCP/IP katman modelleri
- TCP vs UDP, üç-yollu el sıkışma
- IP, port, protokol, bayt sırası

Soket programlama

- POSIX: `socket`, `bind`, `listen`, `accept`, `connect`
- Qt: `QTcpSocket`, `QTcpServer`, sinyal/slot ile asenkron
- Echo sunucu/istemci her iki API ile

Protokol tasarımı

- Length-prefix çerçeveleme
- JSON tabanlı OBYS Protokol v1
- Kendi ve UDP versiyonu

13. Hafta Önizleme: Tam İstemci/Sunucu OBYS

Bölüm 13: OBYS Ağ Sürümü

- Bölüm 11'in thread-safe Depo'su sunucu çekirdeği olur
- OBYS Protokol v1 sunucu ve istemci tarafında uygulanır
- Çoklu istemci, oturum yönetimi
- Hata kurtarma, otomatik yeniden bağlanma
- Bölüm 14: testler, paketleme, dağıtım

Birikim

STL (Bölüm 1-3) + Qt GUI (4-8) + Çok kanallı (9-11) + Ağ (12) = **tam profesyonel OBYS**.

Quiz, Lab ve Kaynaklar

Bu hafta

- **Quiz 12**: OSI, TCP/UDP, soket çağrıları, çerçeveleme
- **Lab 12**: POSIX echo sunucu + Qt echo istemci
- **Çalışma kâğıdı**: OBYS Protokol v1 mesajları yaz

Kaynaklar

- Stevens, "UNIX Network Programming" (klasik)
- Qt 6 belgeleri: `QTcpSocket`, `QTcpServer`
- RFC 793 (TCP), RFC 8259 (JSON)
- Wireshark ile paket inceleme alıştırmaları

Sorularınız?

Bölüm 12 – Ağ Programlamaya Giriş
Yazılım Geliştirme II